

Security Audit Report

Project: Kolekto — POS **Date:** 2026-06-29 **Auditor:** Claude Code (automated, 5-area protocol)
Scope: Auth & Access Control · Privacy Boundaries · API Security · Data Integrity · Production Hygiene
Commit: 1fc50eb (branch feat/docker-registry-deploy)

Executive Summary

The application has a **strong security baseline** — JWT (HS256, no alg-confusion), bcrypt password hashing, rate-limited auth endpoints, locked CORS, Pydantic validation on every route, no SQL injection surface, no committed secrets, soft-delete everywhere, and an append-only audit log. **However, this audit surfaced two CRITICAL financial-integrity defects that were not caught by the prior two audits:** gift-card payments in a sale **never debit the card balance** (unlimited free purchases), and gift-card redemption has **no row lock** (double-redeem race). A cluster of HIGH issues compounds the money risk — returns trust client-supplied prices and have no cumulative-quantity cap (refund inflation), and several write endpoints (suppliers, customer PII) enforce roles only on the client, leaving the API open to any cashier.

The biggest risk is **direct, unbounded financial loss** through gift cards and returns. None of these require privileged access — a logged-in cashier (or anyone with a valid token) can trigger them. Compliance posture is moderate: the technical foundation scores well, but missing audit trails on fraud-sensitive mutations (price changes, stock adjustments, cash sessions) and the absence of TLS in the shipped production config drag the scores down.

Risk Score: 45/100 ● (down from 62/100 — the CRITICAL money bugs outweigh the baseline improvements; resolving C1/C2/H1/H2 returns it to ~70+.)

HIPAA note: This is a retail POS handling no PHI. HIPAA is scored for the overlapping technical safeguards only (access control, audit, transmission security) and is **not materially applicable** to this product.

Compliance Scores

Framework	Score	%	Posture
SOC2 TSC	17 / 28	61%	● Partial — access control & change mgmt solid; monitoring (CC7.2) weakened by missing audit logs
HIPAA	9 / 18	50%	○ Low applicability — no PHI; transmission security (§164.312(e)) absent (no TLS)
CMMC L2	12 / 22	55%	● Partial — IA strong; AU (audit) and SC (transmission/crypto) weak
ISO 27001:2022	14 / 24	58%	● Partial — A.8 controls good on auth/secrets; logging (A.8.15) & crypto-in-transit (A.8.24) gaps

Findings

● **CRITICAL**

[CRITICAL] — Gift-card payment in a sale never debits the card balance

- What: `create_sale` records a `gift_card` payment and stores `gift_card_id`, but never calls `redeem_gift_card`; `GiftCard.current_balance` is never reduced.
- Why it matters: One gift card can pay for an unlimited number of sales. The card stays active with full balance after every "redemption" — direct, unbounded financial loss.
- File/location: `backend/app/services/sale_service.py:256-257` (accumulates `gift_card_total` only; no redemption call anywhere in the sale path). **Verified by manual read.**
- Fix: In the payment loop, when `method == "gift_card"`, call `redeem_gift_card(session, gift_card_id, amount_in_mxn, sale_id=sale.id)` inside the same transaction; fail the sale if redemption fails or balance is insufficient.
- Controls: { soc2: ["CC6.1", "CC7.2"], hipaa: [], cmmc: ["AC.L2-3.1.1"], iso27001: ["A.8.2", "A.8.3"] }

[CRITICAL] — Gift-card redemption has no row lock → double-redeem race (TOCTOU)

- What: `redeem_gift_card` reads balance, checks `amount <= balance`, then writes `balance - amount` with no `SELECT ... FOR UPDATE` and no DB-level non-negative constraint. (`with_for_update` has zero occurrences in the codebase.)
- Why it matters: Two concurrent redemptions both pass the check and both debit, draining more than the balance. Combined with C1, gift cards are effectively unbounded currency.
- File/location: `backend/app/services/gift_card_service.py:156-195` (read @170, check @171, write @181).
- Fix: `select(GiftCard).where(...).with_for_update()` before the balance check, all in one transaction. Add `CHECK (current_balance >= 0)` as a DB backstop.
- Controls: { soc2: ["CC7.2"], hipaa: [], cmmc: ["SC.L2-3.13.1"], iso27001: ["A.8.24"] }

● HIGH

[HIGH] — Returns trust client-supplied `unit_price_mxn` → refund inflation

- What: `create_return` computes refund subtotal from `item_data.unit_price_mxn` (request body), not from the authoritative `SaleItem.unit_price_mxn` already on file.
- Why it matters: A cashier sets an arbitrary price in the return request and receives a cash/gift-card refund far exceeding what was paid.
- File/location: `backend/app/services/return_service.py:130`; schema `backend/app/schemas/extras.py:51-55`.
- Fix: Use the looked-up `sale_item.unit_price_mxn` (already fetched @102) to compute the subtotal; remove `unit_price_mxn` from `ReturnItemCreate`.
- Controls: { soc2: ["CC6.1"], hipaa: [], cmmc: ["AC.L2-3.1.3"], iso27001: ["A.8.2", "A.8.3"] }

[HIGH] — Returns allow repeated full-quantity returns (no cumulative cap)

- What: Per-item check is only `qty_returned <= sale_item.quantity` for the current request; prior returns against the same sale item are not summed.
- Why it matters: Submitting the same return N times refunds Nx the purchase — double/triple refund.
- File/location: `backend/app/services/return_service.py:120-128`.
- Fix: Sum existing `ReturnItem.quantity_returned` for `original_sale_item_id` and enforce `sum + new <= sold`.
- Controls: { soc2: ["CC7.2"], hipaa: [], cmmc: ["AC.L2-3.1.3"], iso27001: ["A.8.24"] }

[HIGH] — Sale stock decrement has no row lock → oversell race

- What: `create_sale` reads `product.stock_quantity`, checks `>= qty`, subtracts — no lock, no DB non-negative constraint.
- Why it matters: Concurrent sales of the same product both pass the check and drive stock negative; inventory integrity loss (lower severity than C2 — inventory, not cash).
- File/location: `backend/app/services/sale_service.py:162-167`.
- Fix: `with_for_update()` on the product row, or atomic `UPDATE ... SET stock = stock - :q WHERE stock >= :q`.
- Controls: { soc2: ["CC7.2"], hipaa: [], cmmc: [], iso27001: ["A.8.24"] }

[HIGH] — Suppliers: any cashier can create/edit/delete suppliers

- What: All `suppliers.py` mutation endpoints gate on `CurrentUser` (authenticated only), not role. Frontend hides suppliers from cashiers (`Sidebar.tsx:55`) but that is client-side only.
- Why it matters: A cashier can POST/PUT supplier records (legal name, payment terms, bank data) with a raw API call using any valid token.
- File/location: `backend/app/routers/suppliers.py:31` (create), `:64` (update).
- Fix: Change `_user: CurrentUser` → `_user: SupervisorUser` on `create_supplier` / `update_supplier`.
- Controls: { soc2: ["CC6.3"], hipaa: ["\$164.312(a)(1)"], cmmc: ["AC.L2-3.1.5"], iso27001: ["A.5.15", "A.8.3"] }

[HIGH] — Customers: any cashier can create/edit customer PII

- What: `customers.py` create/update gate on `CurrentUser` only; cashier write-gating is purely client-side.
- Why it matters: Any cashier can mutate customer PII (name, email, RFC/tax id) via the API.
- File/location: `backend/app/routers/customers.py:43` (create), `:76` (update). Read endpoints are fine for cashier.
- Fix: Gate `create_customer` / `update_customer` on `SupervisorUser` (confirm intended write role with product).
- Controls: { soc2: ["CC6.3"], hipaa: ["\$164.312(a)(1)"], cmmc: ["AC.L2-3.1.5"], iso27001: ["A.5.15", "A.8.3"] }

[HIGH] — Production Caddy config is HTTP-only (no TLS / no auto-HTTPS)

- What: Both Caddyfiles bind `:80` plaintext with no domain block, so Caddy's automatic HTTPS never triggers. `coralslrc.shop` appears nowhere in the repo. The proxy publishes `:80` only.
- Why it matters: JWT bearer tokens, admin login, and customer/sales data travel unencrypted. The HSTS header over HTTP is meaningless.
- File/location: `caddy/Caddyfile:1`, `docker-compose.prod.yml:111`, proxy ports `docker-compose.prod.yml:92-93`.
- Fix: Replace `:80` with the real site address `coralslrc.shop { ... }` so Caddy auto-provisions Let's Encrypt, and expose `443`. If TLS is terminated by an upstream LB, document that and drop the HSTS-over-HTTP header.
- Controls: { soc2: ["CC6.7"], hipaa: ["\$164.312(e)(1)"], cmmc: ["SC.L2-3.13.8"], iso27001: ["A.8.24"] }

MEDIUM

[MEDIUM] — Reports (revenue, margins, exports) readable by cashier

- What: All 11 `reports.py` endpoints gate on `CurrentUser` only; frontend restricts Reports to admin/supervisor.

- Why it matters: Any cashier can pull full revenue/financial reports via the API.
- File/location: `backend/app/routers/reports.py` (all endpoints).
- Fix: Apply a router-level `dependencies=[Depends(require_supervisor_or_admin)]` on the `reports APIRouter`.
- Controls: { soc2: ["CC6.3"], hipaa: ["\$164.312(a)(1)"], cmmc: ["AC.L2-3.1.5"], iso27001: ["A.8.3"] }

[MEDIUM] — IDOR: any user can read any cashier's session by ID

- What: `GET /sales/sessions/{session_id}` returns any session with no ownership/role check (sibling `sessions/current` and `sessions/close` correctly scope to `current_user.id`).
- Why it matters: A cashier can enumerate other cashiers' opening/closing cash balances.
- File/location: `backend/app/routers/sales.py:103-118`.
- Fix: After load, if `session.cashier_id != current_user.id` and `current_user.role` not in `{"admin", "supervisor"}`: raise 403.
- Controls: { soc2: ["CC6.1"], hipaa: ["\$164.312(a)(1)"], cmmc: ["AC.L2-3.1.2"], iso27001: ["A.5.15", "A.8.3"] }

[MEDIUM] — Customer & supplier mutations write no audit log

- What: Create/update/soft-delete of customers and suppliers happen with no `AuditLog` entry.
- Why it matters: PII can be created/alter/deleted with no actor+timestamp trail — fails accountability for personal data.
- File/location: `backend/app/routers/customers.py`, `backend/app/routers/suppliers.py`.
- Fix: Write `AuditLog(actor_id, action="customer.{created,updated,deleted}", entity_type, entity_id)` on each mutation.
- Controls: { soc2: ["CC7.2"], hipaa: ["\$164.312(b)"], cmmc: ["AU.L2-3.3.1"], iso27001: ["A.8.15"] }

[MEDIUM] — Catalog mutations (product, category, stock adjust) write no audit log

- What: `catalog_service` create/update/soft-delete and `adjust_stock` emit no `AuditLog` (stock writes a `StockMovement` ledger row, but not the actor+action audit entry for price/category edits).
- Why it matters: Price changes and manual stock adjustments — the highest-fraud-risk edits in a POS — are untraceable in the audit log.
- File/location: `backend/app/services/catalog_service.py` (all mutations; `adjust_stock @250-329`).
- Fix: Emit `AuditLog` for product/category create/update/delete and stock adjustments (`stock.adjusted` with `delta + reason`).
- Controls: { soc2: ["CC7.2"], hipaa: [], cmmc: ["AU.L2-3.3.1"], iso27001: ["A.8.15"] }

[MEDIUM] — Cashier session open/close writes no audit log

- What: `open_session` / `close_session` set monetary totals and cash difference but emit no `AuditLog`.
- Why it matters: Cash-drawer reconciliation (`difference_mxn`) is fraud-sensitive with no audit trail of who opened/closed and declared what.
- File/location: `backend/app/services/cashier_session_service.py:18-101`.
- Fix: Add `AuditLog` on open and close, including `difference_mxn`.
- Controls: { soc2: ["CC7.2"], hipaa: [], cmmc: ["AU.L2-3.3.1"], iso27001: ["A.8.15"] }

[MEDIUM] — Missing HTTP security headers in production (Permissions-Policy, HSTS) + weak CSP

- What: `Permissions-Policy` is absent everywhere; HSTS is in the dev `caddy/Caddyfile` but absent from the shipped `docker-compose.prod.yml` embedded config; CSP allows `'unsafe-inline'` / `'unsafe-eval'` on scripts.
- Why it matters: Camera/mic/geolocation/USB ungated; weakened XSS defense; no transport-security pinning in prod.
- File/location: `docker-compose.prod.yml:121-128` , `caddy/Caddyfile:23-24` , `frontend/next.config.ts:3-20` .
- Fix: Add `Permissions-Policy` `"camera=(), microphone=(), geolocation=(), usb=()"` and `Strict-Transport-Security` `"max-age=31536000; includeSubDomains"` to the prod header block; nonce inline scripts to drop `'unsafe-inline'` .
- Controls: { soc2: ["CC6.6", "CC6.7"], hipaa: ["§164.312(e)(1)"], cmmc: ["SC.L2-3.13.8"], iso27001: ["A.8.23", "A.8.26"] }

[MEDIUM] — Postgres `sslmode` not set

- What: The DB URL is built from components with no `sslmode` / `ssl` parameter.
- Why it matters: App→DB connections default to non-SSL. Mitigated today by same-host Docker networking, but breaks the `sslmode=require` standard and fails if the DB is ever externalized.
- File/location: `backend/app/config.py:30-34` .
- Fix: Append `?ssl=require` (asyncpg) / `?sslmode=require` (sync) when `db_host` is not loopback/internal, or make it a flag.
- Controls: { soc2: ["CC6.7"], hipaa: ["§164.312(e)(1)"], cmmc: ["SC.L2-3.13.8"], iso27001: ["A.8.24"] }

[MEDIUM] — Financial-history FKs lack explicit `ON DELETE RESTRICT`

- What: `sales.customer_id → customers.id` and `sales.cashier_id → users.id` have no `onDelete` (NO ACTION). Safe today because the app only soft-deletes, but a future hard-delete endpoint or manual DB delete would orphan/break financial history.
- Why it matters: Defense-in-depth gap; integrity relies entirely on the app always soft-deleting. (Extras tables correctly use `ON DELETE RESTRICT` — good.)
- File/location: `backend/alembic/versions/20260506_001004_operations_tables.py:105-107` ; `...001003_people_tables.py:127` .
- Fix: Make financial-history FKs explicit `ON DELETE RESTRICT` .
- Controls: { soc2: ["CC6.1"], hipaa: [], cmmc: [], iso27001: ["A.8.10"] }

LOW

[LOW] — Gift-card code logged in plaintext at issue/redeem

- What: `log.info("gift_card.issued", code=code)` and the redeem log expose the full redeemable code.
- Why it matters: The code is a bearer instrument; anyone with log access can redeem it (compounded by C1/C2).
- File/location: `backend/app/services/gift_card_service.py:116-121` , `:197-203` .
- Fix: Log only `gift_card_id` , never `code` .
- Controls: { soc2: ["CC6.1"], hipaa: ["§164.312(b)"], cmmc: [], iso27001: ["A.8.12"] }

[LOW] — `SECRET_KEY` has no minimum-length validation

- What: JWT signing key is `Field(...)` with no length floor — a 4-char key is accepted.
- Why it matters: A short/guessable HS256 key allows JWT forgery → full auth bypass.
- File/location: `backend/app/config.py:36` .

- Fix: `secret_key: str = Field(..., min_length=32, alias="SECRET_KEY")` ; reject known defaults when `env == "production"` .
- Controls: { soc2: ["CC6.1"], hipaa: ["\$164.312(a)(2)(iv)"], cmmc: ["IA.L2-3.5.7"], iso27001: ["A.8.24"] }

[LOW] — Default admin password is a code constant, only warned (not blocked) in prod

- What: `ADMIN_INITIAL_PASSWORD` defaults to `Admin123!` ; startup logs a warning but still boots.
- Why it matters: A prod deploy left on the default has a known admin credential.
- File/location: `backend/app/config.py:73` , `backend/app/main.py:24,30-35` .
- Fix: In `lifespan` , refuse to start when `is_production` and the admin password equals the default.
- Controls: { soc2: ["CC6.1"], hipaa: ["\$164.308(a)(5)(ii)(D)"], cmmc: ["IA.L2-3.5.7"], iso27001: ["A.5.17"] }

[LOW] — `/openapi.json` exposed in production

- What: `docs_url` / `redoc_url` are disabled in prod but `openapi_url` is left at default.
- Why it matters: The raw OpenAPI schema (all routes, params, models) is publicly served — full API enumeration.
- File/location: `backend/app/main.py:42-47` .
- Fix: `openapi_url=None` if `settings.is_production` else `"/openapi.json"` .
- Controls: { soc2: ["CC6.1"], hipaa: [], cmmc: ["SC.L2-3.13.1"], iso27001: ["A.8.26"] }

[LOW] — `backend/.env.test` is tracked despite being gitignored

- What: File is committed (in `3d5705a`) though listed in `.gitignore:34` . Contents are **dummy values only** (localhost DSNs, `test_secret_key_...`) — no real secret leaked.
- Why it matters: A gitignored-yet-tracked file is a footgun — the ignore rule is silently ineffective, so a future real secret added here would be committed unnoticed.
- File/location: `backend/.env.test:1-11` .
- Fix: `git rm --cached backend/.env.test` . No secret rotation needed.
- Controls: { soc2: ["CC6.1"], hipaa: [], cmmc: ["SC.L2-3.13.1"], iso27001: ["A.8.12"] }

[LOW] — Sales list/read not scoped to cashier

- What: `GET /sales` and `GET /sales/{id}` return all sales to any authenticated user.
- Why it matters: A cashier sees all sales history, not just their own. Plausibly by design for a single-store POS — confirm intent.
- File/location: `backend/app/routers/sales.py:178` , `:198` .
- Fix: If cashiers should see only their own, filter by `current_user.id` for non-supervisor roles; otherwise document as intended.
- Controls: { soc2: ["CC6.3"], hipaa: [], cmmc: [], iso27001: ["A.8.3"] }

Verified Clean (no findings)

- **JWT/auth core:** HS256 pinned (no alg-confusion), 60-min expiry, `SECRET_KEY` required (no insecure default), bcrypt, login + change-password rate-limited (10/min), `get_current_user` checks `deleted_at IS NULL` + `is_active` .
- **Role enforcement (correct):** `users.py` (admin-gated, own-record-or-admin, mass-assignment of role/is_active blocked, self-delete blocked), `catalog.py` (mutations admin, stock-adjust supervisor), `settings.py` (admin), `purchases.py` (supervisor), `extras.py` gift-card/return creation (supervisor).

- **Input validation:** Pydantic v2 schema or bounded `Query /path` params on every DB-touching route; no raw `dict` bodies.
- **SQL injection:** No raw user input in SQL; `text()` only injects a fixed tenant literal; `ilike` builds bound parameters.
- **Secret exposure:** `UserRead` excludes `password_hash` ; no JWT secret or hash in any response.
- **CORS:** Locked to env-driven origins, no wildcard.
- **File uploads:** None (logo/favicon are HTTPS-validated URL strings).
- **Ownership columns:** `actor_id` / `cashier_id` / `created_by` set server-side from the authenticated user, never from request bodies; gift-card codes server-generated.
- **Sale totals:** Computed server-side from looked-up product prices (only the `return` path trusts client price — H1).
- **Soft deletes:** Users, products, categories, customers, suppliers, purchases all soft-delete; no hard `session.delete()` in app code.
- **Audit-log immutability:** No update/delete path for `audit_logs` — insert-only.
- **Docker hardening:** Non-root `USER` , pinned slim base images, no secrets baked into layers, Postgres `5432` not published to host.
- **Production mode:** No `debug=True` , no `--reload` ; frontend runs `NODE_ENV=production` standalone; `/docs` and `/redoc` disabled in prod.
- **Logging:** No passwords, tokens, request bodies, or customer PII logged (except gift-card code — L1).
- **No secrets committed** (only `.env.example` + dummy `.env.test`); no secrets in git history.
- **TypeScript:** `tsc --noEmit` passes clean (0 errors).

Remediation Priority

1. **C1 — Debit gift-card balance on sale redemption.** Stops unlimited free purchases. (Highest \$ impact, ~10 lines.)
2. **C2 — `with_for_update()` on gift-card redemption + `CHECK (current_balance >= 0)` .** Stops double-redeem.
3. **H1 — Use server-side `sale_item.unit_price_mxn` for refunds;** drop client price from the schema.
4. **H2 — Cap cumulative return quantity** against prior returns.
5. **H3 — Lock product row (or atomic conditional UPDATE) on stock decrement.**
6. **HIGH access gaps — swap `CurrentUser` → `SupervisorUser`** on suppliers + customers mutations; router-level guard on reports; ownership check on session-by-id (one-line each).
7. **HIGH transport — Caddy `coralslrc.shop { ... }` with auto-HTTPS + expose 443** (or document upstream TLS).
8. **MEDIUM audit logs — emit `AuditLog`** on customer/supplier, catalog, stock-adjust, and cashier-session mutations.
9. **MEDIUM hardening — add Permissions-Policy + HSTS to prod headers; set Postgres `sslmode` ; explicit FK ON DELETE RESTRICT .**
10. **LOW cleanup — stop logging gift-card codes; `SECRET_KEY` min_length; block default admin password in prod; disable `/openapi.json` in prod; `git rm --cached backend/.env.test` .**

Items 1–5 are the production blockers. They are all small, localized diffs in `sale_service.py` , `gift_card_service.py` , and `return_service.py` . Recommend fixing 1–7 before the VPS deploy and re-running this audit to confirm the score returns above 70.